MACHINE LEARNING PROJECT

Vendor Invoice Risk Intelligence System

AI-Driven Freight Cost Prediction & Invoice Risk Flagging

Project Overview

This document is a comprehensive technical overview of the Vendor Invoice Risk Intelligence System — a machine learning system designed to automate two high-value finance operations: predicting freight costs on vendor invoices and identifying invoices that carry financial or operational risk.

Project	Vendor Invoice Risk Intelligence System
Domain	Supply Chain and Finance Operations
ML Type	Supervised Learning — Regression and Classification
Pipelines	Freight Cost Prediction + Invoice Risk Flagging
Interface	Streamlit Web Application
Database	SQLite — inventory.db (vendor_invoice + purchases)

1. Executive Summary

The Vendor Invoice Risk Intelligence System addresses two persistent problems in procurement finance: freight costs that are difficult to forecast and invoice anomalies that consume disproportionate manual review time. The system replaces both with trained machine learning models that run automatically, consistently, and at scale.

3 Regression Models Evaluated per training run Best selected by MAE	**1,080 Model Fits** GridSearchCV across 216 hyperparameter sets	**Streamlit App** Unified web interface No code required

What This System Delivers

- A regression pipeline that learns the relationship between invoice dollar value and freight cost from historical data, enabling finance teams to forecast and validate logistics charges automatically.
- A classification pipeline that scores every invoice for risk — flagging those with cost discrepancies or delivery anomalies for human review, and returning a safe result for clean invoices.
- A unified Streamlit web interface that puts both models in the hands of non-technical users with no Python knowledge required.

2. Problem Statement

2.1 The Business Challenge

Procurement teams in mid-to-large organisations process thousands of vendor invoices every month. Two recurring inefficiencies create significant financial risk and operational drag:

Unpredictable Freight Costs

Freight charges on vendor invoices vary with invoice value. Without a reliable data-driven baseline, finance teams cannot tell whether a freight charge is reasonable or inflated — making budget forecasting and vendor negotiations reliant on guesswork.

Manual Invoice Review Bottleneck

Manually reviewing every invoice for dollar discrepancies, quantity mismatches, and delivery delays is slow, inconsistent, and unscalable. As transaction volume grows, the backlog grows with it — creating payment delays, audit risk, and staff burnout.

2.2 The Solution

How Machine Learning Solves Both Problems

- The freight prediction model learns historical patterns between invoice dollar values and freight charges, providing a data-driven baseline for every new invoice.
- The invoice risk classifier learns the signature of problematic invoices — dollar discrepancies, delivery delays, PO mismatches — and scores each new invoice automatically.
- Both models are served through a single Streamlit interface, accessible to anyone in the finance or procurement team without writing a line of code.

3. Project Architecture

3.1 System Architecture

Two independent training pipelines share a common SQLite data source and write serialised models to a shared models/ directory. A unified inference layer serves both models through the Streamlit web application:

3.2 Folder Structure

Each concern — assets, data, training, inference, models, notebooks, app — lives in its own directory with no circular dependencies:

```
Vendor Invoice Risk Intelligence System/
│
├── assets/
│   ├── freight_cost_prediction.PNG
│   └── invoice_flagging.PNG
│
├── data/
│   └── inventory.db
│
├── freight_cost_prediction/
│   ├── data_preprocessing.py
│   ├── model_evaluation.py
│   └── train.py
│
├── inference/
│   ├── predict_freight.py
│   └── predict_invoice_flag.py
│
├── invoice_flagging/
│   ├── data_preprocessing.py
│   ├── modeling_evaluation.py
│   └── train.py
│
├── models/
│   ├── predict_flag_invoice.pkl
│   ├── predict_freight_model.pkl
│   └── scaler.pkl
│
├── notebooks/
│   ├── Invoice Flagging.ipynb
│   └── Predicting Freight Cost.ipynb
│
└── app.py
```

4. Machine Learning Pipelines

Pipeline 1 — Freight Cost Prediction

This pipeline frames freight cost estimation as a supervised regression problem. Each vendor invoice in the training set carries a known freight charge, allowing the model to learn the underlying relationship between invoice value and logistics cost.

Input Feature

Feature	Type	Description
Dollars	Continuous — Numeric	Total dollar value of the vendor invoice

Models Evaluated

Model	Approach	Role in Pipeline
Linear Regression	Parametric	Establishes a linear baseline — fast and interpretable
Decision Tree Regressor	Non-parametric	Captures threshold-based non-linearities, max_depth=5
Random Forest Regressor	Ensemble	Reduces variance by averaging many trees, max_depth=6

Selection Criterion

■ After all three models train on the same split, the one with the lowest Mean Absolute Error (MAE) on the held-out test set is serialised as predict_freight_model.pkl. This ensures the model that generalises best to unseen invoices is the one saved and loaded by the application.

Pipeline 2 — Invoice Risk Flagging

This pipeline frames invoice anomaly detection as a binary classification problem. Rather than reviewing every invoice manually, the model learns what a problematic invoice looks like — based on dollar discrepancies, delivery delays, and PO-level purchasing patterns — and assigns a risk flag automatically.

Feature Engineering (SQL JOIN across two tables)

Feature	Source	What It Captures
invoice_quantity	vendor_invoice	Quantity billed on this invoice
invoice_dollars	vendor_invoice	Dollar amount billed on this invoice
Freight	vendor_invoice	Freight charge on this invoice
total_item_quantity	purchases	Total quantity ordered across all PO lines
total_item_dollars	purchases	Total value ordered across all PO lines — compared to invoice dollars to detect discrepancies

Risk Labelling Logic

An invoice receives a risk flag (label = 1) if either condition is true:

- Condition A — Dollar Discrepancy: |invoice_dollars - total_item_dollars| > $5
- Condition B — Delivery Delay: average receiving delay across PO lines > 10 days

5. Web Application & Screenshots

The two trained models are exposed through a single Streamlit application that finance and procurement teams can use directly in their browser. No Python knowledge is required — users fill in a form and receive an instant prediction.

5.1 Module Overview

Module 1 — Freight Cost Prediction

Input: Invoice dollar value

Output: Predicted freight cost in USD

Use Case: Validate logistics charges, support budget forecasting, inform vendor negotiations

Module 2 — Invoice Risk Flagging

Input: Invoice qty, dollars, freight, total PO qty, total PO dollars

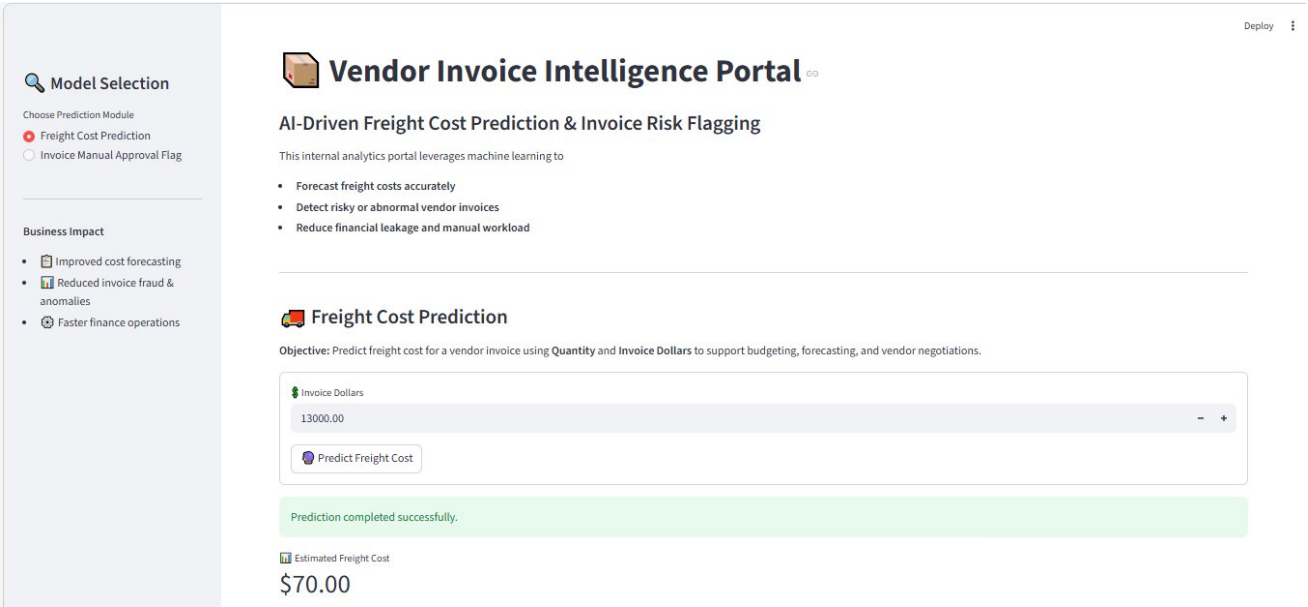
Output: Prediction completed successfully or Invoice requires MANUAL APPROVAL

Use Case: Automated invoice triage, overpayment prevention, audit prioritisation

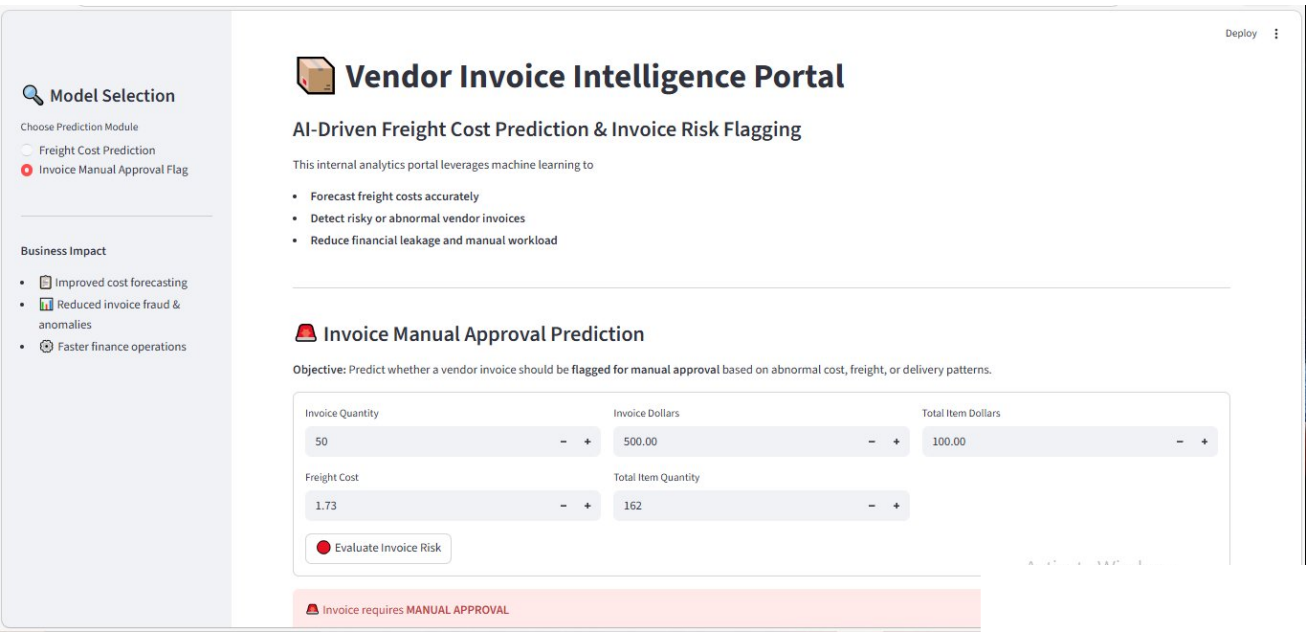
5.2 Application Screenshots

The screenshots below show the live application interface for each prediction module.

Module 1 — Freight Cost Prediction



Module 2 — Invoice Risk Flagging



5.3 Application Output States

The Invoice Risk Flagging module returns one of two clearly labelled output states:

Output State	Meaning	Recommended Action
Safe for Auto-Approval	No discrepancy or delay signals detected. Invoice matches PO data within acceptable thresholds.	Process payment automatically — no manual review required.
Manual Approval Required	Dollar discrepancy exceeds $5 vs PO value, or average receiving delay exceeds 10 days on associated PO lines.	Route to finance reviewer for investigation before payment is authorised.

6. How to Run the Project

Prerequisites

- Python 3.9+
- pip
- VS Code or any terminal

Install Dependencies

```
pip install pandas scikit-learn joblib streamlit
```

Execution Order

Step	Command	What It Does
1	python freight_cost_prediction/train.py	Trains all 3 regression models, selects best by MAE, saves predict_freight_model.pkl
2	python invoice_flagging/train.py	Runs GridSearchCV (1,080 fits), saves best classifier + scaler.pkl
3	python inference/predict_freight.py	Smoke test — prints a sample prediction DataFrame
4	python -m streamlit run app.py	Launches the web app at http://localhost:8501

Before You Run

- Steps 1 and 2 must complete before launching the app — the app loads .pkl files at startup.
- Run all commands from the project root, not from inside a subdirectory.
- Step 2 may take 10-20 minutes on first run — GridSearchCV evaluates 216 parameter combinations across 5 folds.
- If you replace any corrected source file, delete the old .pkl files and retrain from scratch.

7. Business Impact

The Vendor Invoice Risk Intelligence System targets two concrete inefficiencies in the invoice processing lifecycle. By automating freight cost estimation and invoice risk scoring, it reduces the manual effort required to process and validate vendor invoices.

Impact Area	Description
Faster Invoice Review Decisions	Automated risk scoring gives reviewers an immediate model-based signal on every invoice, reducing the time spent deciding which invoices need attention.
Reduced Overpayment Risk	Dollar discrepancy detection catches billing errors and overbillings before payment is authorised — protecting the organisation's bottom line.
Reliable Freight Benchmarking	Finance teams gain a data-driven freight baseline for every invoice, making it possible to challenge abnormal logistics charges with evidence.

Strategic Value

- Consistency: ML applies identical criteria to every invoice — no reviewer fatigue or subjective bias.
- Extensibility: Both models can be retrained on new data as vendor patterns evolve, without changing the application layer.
- Separation of concerns: Each pipeline, inference script, and the app are independent modules — training, updating, or replacing one does not require changes to the others.

8. Challenges & Solutions

Three distinct layers of challenges shaped the development of this system — from initial scoping, through implementation, to deployment.

Challenge 01 — Understanding the Problem — What to Build and What to Ignore	
Challenge	The database contained far more fields than the models needed. Determining which signals genuinely predicted freight cost or invoice risk — and which were noise — required careful analysis before any code was written. Setting the right labelling thresholds ($5 discrepancy, 10-day delay) meant examining value distributions to ensure the flagged class was meaningful, not trivial or overwhelming.
Solution	Exploratory analysis in Jupyter notebooks was used to validate feature correlations and class balance before committing to the feature set and labelling logic.

Challenge 02 — Coding — Silent Failures in the Inference Layer	
Challenge	Two issues produced no errors yet delivered wrong results. First, the invoice classifier was trained on StandardScaler-normalised features but the inference script passed raw unscaled inputs — the shapes matched so scikit-learn raised nothing, but every prediction was based on a shifted distribution. Second, the Streamlit freight form passed two inputs (Quantity + Dollars) to a model trained on one (Dollars only), raising a ValueError on every prediction — making the module completely non-functional from launch.
Solution	The scaler was made part of the inference contract — loaded and applied alongside the model on every prediction call. The form was corrected to match the training feature schema exactly. End-to-end testing from form input to model output became the verification method.

Challenge 03 — Pipeline Management — Consistency Across Two Independent Pipelines	
Challenge	Both pipelines used hardcoded relative paths that only worked from specific directories. Launching the app normally with python -m streamlit run app.py from the project root broke all model file references immediately. A secondary issue: the scaler save function accepted a path argument but ignored it, writing to a hardcoded location — breaking cross-pipeline consistency silently.
Solution	All paths were anchored to each script's own filesystem location using Path(__file__).resolve(). The scaler save bug was fixed so the function honours its argument. A defined training sequence (freight → classifier → app) was documented to make dependency order explicit.

9. Technology Stack

The following technologies were used across all layers of the system:

Category	Technology	Role in Project
Language	Python	All pipelines, inference scripts, and Streamlit app
Data Layer	pandas + sqlite3	SQL JOIN queries, DataFrame manipulation, feature prep
ML Framework	scikit-learn	Model training, GridSearchCV, StandardScaler, metrics
Serialisation	joblib	Persist and reload trained models and scalers as .pkl
Web App	Streamlit	Interactive browser UI — no frontend code required
IDE	VS Code	Development, debugging, integrated terminal

Closing Note

The Vendor Invoice Risk Intelligence System demonstrates that meaningful automation in finance operations does not require complex infrastructure — it requires clearly defined problems, disciplined feature engineering, and a deployment approach that treats training and inference as a single end-to-end contract. Both pipelines are fully functional, retrain-able, and structured to scale as data volumes and business requirements grow.